

LEARNING RESOURCE GUIDE

ISC2 CCSP: Certified Cloud Security Professional (Aug 2026) | Professional Participant Workbook

Learning Objectives

By the end of this lecture and its accompanying guide, you will be able to:

- By the end of this lecture, you will be able to define Infrastructure as a Service, Platform as a Service, and Software as a Service, and accurately describe the security responsibilities that fall to the customer and to the CSP under each model.
- By the end of this lecture, you will be able to explain the shared responsibility model and identify where the responsibility line sits in IaaS, PaaS, and SaaS environments.
- By the end of this lecture, you will be able to describe the 'responsibility gap' — the most common failure mode in cloud security — and explain the conditions under which it occurs.
- By the end of this lecture, you will be able to select the appropriate security controls for a given service category and articulate why controls effective in one category may be irrelevant or insufficient in another.
- By the end of this lecture, you will be able to apply the service category framework to cloud security assessments, compliance conversations, contract reviews, and incident accountability analysis.

Introduction

The three cloud service categories — Infrastructure as a Service, Platform as a Service, and Software as a Service — are not merely marketing labels. They are a framework for understanding who is responsible for securing what. The shared responsibility model, which every major cloud provider publishes and which every major cloud compliance framework references, changes materially depending on which service category is in use. In IaaS, the responsibility line sits just above the virtualisation layer. In PaaS, it sits above the operating system and runtime. In SaaS, it sits above the application itself. Misidentifying which category you are working in is one of the most consistent causes of cloud security incidents.

The 2019 Capital One breach illustrates this directly. An attacker exploited a misconfigured Web Application Firewall and server-side request forgery to retrieve AWS IAM credentials and exfiltrate over 100 million customer records from S3 storage. The infrastructure — the hardware, the data centre, the virtualisation layer — performed exactly as intended. AWS met every security obligation it had. The failure was entirely in the customer layer: WAF configuration, IAM permissions, and storage bucket access policies. All of these are IaaS customer responsibilities. The shared responsibility model was clear. The organisation either did not fully understand it or failed to operationalise it. That gap is what this lecture is designed to close.

Understanding the service categories also has direct implications beyond security operations. It shapes how compliance requirements apply (what does HIPAA require of a SaaS relationship versus an IaaS relationship?), how vendor contracts should be structured, how security assessments should be scoped, and how incident accountability is determined. For a CCSP candidate, the service category framework is foundational — it is assumed knowledge in virtually every other domain the exam covers.

Core Concepts Explained

Infrastructure as a Service (IaaS) and Customer Security Responsibility

Definition: In IaaS, the cloud service provider manages the physical hardware, data centre facility, network infrastructure, and virtualisation layer. The customer receives virtual machines, storage, and networking and is fully responsible for everything above the hypervisor: the operating system, middleware, runtime, application code, and data.

Business relevance: IaaS provides the greatest customer control over the environment and the highest customer security responsibility. Organisations that choose IaaS for flexibility must also commit to the operational discipline of patching, hardening, and monitoring everything they deploy on that infrastructure. Failure to do so is the most common cause of cloud security incidents.

Cloud Security Practitioner relevance: Cloud Security Practitioners working in IaaS environments are responsible for OS patch management, host-based firewall configuration, endpoint protection, network security group rules, storage encryption and access control policies, and identity and access management for infrastructure operations. These controls cannot be delegated to the CSP.

Real-world example: The 2019 Capital One breach is the canonical IaaS shared responsibility failure. Capital One was running an EC2-based environment (IaaS) on AWS. AWS's responsibilities — physical security, hypervisor integrity, network hardware — were all met. Capital One's responsibilities — WAF configuration, IAM permission scoping, S3 bucket access policies — were not. The WAF was misconfigured to allow server-side request forgery, the IAM role attached to the WAF had excessive S3 permissions, and the S3 buckets themselves were not protected with additional access controls that would have detected or blocked the unusual read pattern. Each of these failures represents an IaaS customer control that was not properly implemented. The practitioner's lesson: in IaaS, every configuration decision above the hypervisor is the customer's responsibility, and 'the cloud handles it' is never a valid assumption for any control in that layer.

Risks or limitations: The responsibility gap in IaaS is wide. Customers often underestimate the operational overhead of managing operating system lifecycle, patch currency, and configuration compliance across a dynamically scaling IaaS environment. The best practice is to implement automated configuration management (using tools such as AWS Systems Manager, Chef, or Puppet), enforce mandatory OS hardening baselines at instance launch, and run continuous compliance scanning to detect drift from those baselines.

Platform as a Service (PaaS) and the Shifted Security Boundary

Definition: In PaaS, the cloud service provider manages the operating system, runtime environment, and much of the underlying middleware. The customer focuses on application code and data. The customer does not install or patch operating systems, manage runtime engines, or configure infrastructure-level load balancers.

Business relevance: PaaS reduces the customer's operational burden for infrastructure and OS management, but it introduces a new dependency on the provider's runtime environment. If a vulnerability exists in the platform's runtime that the CSP fails to patch, the CSP is responsible. If the customer's application code is poorly written, that is entirely the customer's problem. The attack surface shifts — it narrows at the infrastructure layer and opens at the application layer.

Cloud Security Practitioner relevance: In PaaS environments, Cloud Security Practitioners focus on application security testing (SAST, DAST), secrets management (ensuring credentials are not hardcoded or exposed in environment variables), API security, dependency scanning for vulnerable libraries in application code, and identity management at the application level. Infrastructure hardening is substantially the provider's domain.

Real-world example: Heroku, Google App Engine, and Azure App Service are PaaS platforms used extensively in enterprise development. A software company running its SaaS product on Heroku (a PaaS) experienced a security incident in which an attacker extracted API credentials embedded directly in the application's source code — credentials that had been accidentally committed to the application's Git repository. The PaaS platform itself was secure: Heroku's runtime, networking, and OS were current and properly configured. The vulnerability was in the customer's application code — a secrets management failure entirely within the customer's responsibility. The cloud security practitioner's post-incident review recommended: mandatory pre-commit hooks to scan for credential patterns in code, centralised secrets management using HashiCorp Vault or the CSP's secrets manager, and automated SAST scanning in the CI/CD pipeline. These are all PaaS-layer customer controls, distinct from the infrastructure controls that would apply in an IaaS engagement.

Risks or limitations: The most common PaaS security failure is treating the shift in responsibility as a reduction in overall security work rather than a redirection of it. Customers who move from IaaS to PaaS and reduce their security investment proportionally often find that application-layer vulnerabilities — injection flaws, broken authentication, insecure direct object references — become the dominant risk. The best practice is to implement a robust application security programme (OWASP Top 10 coverage, dependency scanning, secrets management) proportional to the risk, regardless of the infrastructure model in use.

Software as a Service (SaaS) and Configuration-Focused Security

Definition: In SaaS, the customer consumes a finished application delivered over the internet. The CSP manages the application, the platform it runs on, and all underlying infrastructure. The customer's security responsibilities narrow to identity and access management, data governance within the application, and configuration of the security features the provider makes available.

Business relevance: SaaS misconfigurations are responsible for some of the highest-impact data exposures in recent years. The customer cannot modify the application code or the infrastructure, but they can expose sensitive data through incorrect sharing permissions, over-provisioned access rights, or failure to enable available security features such as multi-factor authentication and data loss prevention policies.

Cloud Security Practitioner relevance: Cloud Security Practitioners in SaaS environments focus on: reviewing and enforcing access control configurations (role-based access, least privilege, MFA enforcement), classifying data stored within the application and applying appropriate data governance policies, enabling all available security features (audit logging, DLP, session controls), and reviewing the provider's data handling policies, data residency commitments, and breach notification procedures.

Real-world example: Microsoft 365 is one of the most widely deployed SaaS platforms in the world. In 2023, Microsoft disclosed that a threat actor referred to as Storm-0558 had accessed email accounts belonging to US government agencies through Microsoft 365 by forging authentication tokens using a compromised Microsoft signing key — a failure in Microsoft's own application security (the CSP's responsibility). However, investigations also revealed that affected organisations had not enabled audit logging for their M365 tenants, which would have provided earlier detection of the anomalous access patterns. Enabling audit logging in M365 is a customer configuration responsibility — it is a security feature the provider makes available, but the customer must turn it on. This incident illustrates both sides of SaaS shared responsibility: the application security failure was Microsoft's, but the detection gap was the customers'. Practitioners should audit all available security features in every SaaS platform and ensure that every available feature relevant to their risk profile is enabled and configured correctly.

Risks or limitations: SaaS misconfiguration risk is significant precisely because the configurations are often managed by business users rather than security professionals. Default settings in major SaaS platforms are frequently permissive. The best practice is to establish a SaaS security configuration baseline for each deployed

platform (using the CIS Benchmarks for Microsoft 365, Google Workspace, or Salesforce where available), review configurations on a defined schedule, and use a Cloud Access Security Broker (CASB) to provide continuous visibility into configuration state across the SaaS portfolio.

The Shared Responsibility Model and the Responsibility Gap

Definition: The shared responsibility model is the formal framework through which cloud providers describe the division of security obligations between themselves and their customers. The model shifts depending on service category. The 'responsibility gap' is the space between what customers assume the provider covers and what the provider actually covers — it is the most common proximate cause of cloud security incidents.

Business relevance: Every major CSP publishes a shared responsibility model document. Understanding this document for each specific service in use is not optional — it is foundational to cloud security governance. The responsibility gap most frequently appears in IaaS environments where customers assume infrastructure security extends to their operating systems and applications, but it also appears in SaaS environments where customers assume configuration security is handled by the provider.

Cloud Security Practitioner relevance: Cloud Security Practitioners are responsible for translating the shared responsibility model into operational security procedures for every service in the organisation's cloud portfolio. This includes reviewing provider documentation for each service, mapping provider responsibilities to customer responsibilities, and ensuring that all customer-side obligations are addressed by named controls and named owners.

Real-world example: A healthcare technology company migrated from on-premises servers to AWS EC2 (IaaS) to reduce infrastructure costs. The project team assumed that 'moving to the cloud' meant the cloud provider would handle security patching. AWS's shared responsibility model document explicitly states that operating system patching is the customer's responsibility in EC2. Within eight months of migration, an attacker exploited a known vulnerability in an unpatched Linux kernel on one of the EC2 instances to gain root access. The resulting breach required HIPAA breach notification for 47,000 patients. The responsibility gap — the distance between what the team assumed and what AWS actually covered — was the direct cause. Post-incident, the organisation implemented AWS Systems Manager Patch Manager to automate OS patching across all EC2 instances, enforced patching via Service Control Policies, and added patch compliance to its monthly security reporting. The gap was closed, but at significant reputational and financial cost.

Risks or limitations: The responsibility gap widens whenever service categories are mixed without explicit mapping of the shifting responsibility line. The best practice is to document the responsibility line for every service in use, review it whenever services are added or changed, and require explicit sign-off from the CISO that all customer-side responsibilities have a named owner and an implemented control.

Multi-Tenancy Security Implications Across Service Categories

Definition: Multi-tenancy means that multiple customer organisations share the same underlying cloud infrastructure or application instance. The security implications of multi-tenancy differ materially by service category: in IaaS the primary concern is hypervisor isolation, in PaaS it is runtime isolation, and in SaaS it is application-level data partitioning.

Business relevance: Multi-tenancy is fundamental to cloud economics — shared infrastructure is what makes cloud services affordable. Understanding the isolation mechanisms and their failure modes is essential for assessing the risk of any cloud deployment and for making informed decisions about whether multi-tenant or dedicated offerings are appropriate for a given workload.

Cloud Security Practitioner relevance: Cloud Security Practitioners evaluate multi-tenancy risk by: reviewing provider documentation for the isolation mechanisms in use at each layer, assessing the organisation's risk

tolerance for shared infrastructure, identifying workloads that require dedicated infrastructure due to regulatory or contractual requirements, and verifying that access controls within shared environments (especially SaaS) correctly partition data between tenants.

Real-world example: In 2022, a vulnerability in a popular cloud storage SaaS platform was found to allow users to access other tenants' files under specific conditions by manipulating API parameters — a classic broken object-level authorisation failure in a multi-tenant application. The SaaS provider was responsible for the application-level data partitioning, and the failure was unambiguously theirs. However, customers discovered the incident through their own audit logs — organisations that had enabled audit logging and anomaly detection in the platform's activity logs detected unusual API patterns before the provider disclosed the incident. This underscores two lessons: first, evaluate the provider's multi-tenancy isolation controls during vendor assessment (request information about their data partitioning architecture and penetration testing scope); second, enable all available audit logging in SaaS platforms so that anomalous access patterns are detectable from the customer side, even when the failure is the provider's.

Risks or limitations: The most severe multi-tenancy risk is tenant isolation failure — a vulnerability in the hypervisor (IaaS), runtime (PaaS), or application (SaaS) that allows one tenant to access another's resources. The best practice is to review the CSP's published information on isolation mechanisms, request independent penetration testing results or SOC 2 reports that cover tenant isolation controls, and for high-sensitivity workloads, evaluate dedicated instance or private cloud options.

Practical Applications

The following scenarios show how the service category framework guides Cloud Security Practitioners through real operational decisions — from incident analysis to contract negotiation to security control design.

Applying the Shared Responsibility Model During a Cloud Migration Assessment

A manufacturing organisation is planning to migrate its enterprise resource planning (ERP) system from on-premises infrastructure to the cloud. The project team is evaluating two options: deploying the ERP application on AWS EC2 (IaaS) or subscribing to a SaaS-delivered version of the same ERP from the software vendor. The Cloud Security Practitioner is asked to assess the security implications of both options. For the IaaS option, the practitioner documents the full list of customer-side security responsibilities: OS hardening and patching, database encryption configuration, network security group rules, backup and recovery, and application-layer access controls. For the SaaS option, the practitioner narrows the customer responsibility list to: identity and access management configuration, data export and portability requirements, audit logging enablement, and the vendor's own security certifications and breach notification commitments. The practitioner presents both responsibility maps to the CISO with an honest assessment of the organisation's operational capacity: the IaaS option provides more control but requires a larger security operations investment; the SaaS option narrows the responsibility surface but creates a dependency on the vendor's security posture. The CISO uses this analysis to make an informed architectural decision — and in this case chooses the SaaS option because the organisation lacks the OS patching discipline to manage IaaS safely.

Scoping a Cloud Security Assessment by Service Category

A Cloud Security Practitioner is engaged to conduct a security assessment of a retail company's cloud environment. The environment includes three distinct service categories: AWS EC2 instances running the core e-commerce application (IaaS), AWS Lambda functions handling inventory sync (PaaS), and Salesforce managing customer relationship data (SaaS). The practitioner scopes each assessment tier separately. For the EC2 tier, the assessment includes OS patch currency, host-based firewall rules, encryption at rest for attached volumes, and

IAM roles assigned to each instance. For the Lambda tier, the assessment focuses on function code security (injection vulnerabilities, secrets in environment variables), IAM execution role permissions, and dependency library vulnerability scanning. For the Salesforce tier, the assessment covers sharing rules and profile permissions, field-level security settings, audit log retention configuration, and the company's Salesforce Shield Event Monitoring enablement. The practitioner produces three separate findings tables, each labelled by service category, and presents them to the IT leadership team with a clear explanation of why the control set differs across all three environments. This structured approach prevents the common mistake of applying one assessment methodology to all cloud services regardless of category.

Designing a DevSecOps Pipeline That Reflects Service Category Responsibilities

A software development organisation is building a new cloud-native application on a PaaS environment using Google App Engine. The Cloud Security Practitioner is asked to help design the DevSecOps pipeline. Because the team is using PaaS, OS and runtime patching is Google's responsibility — the pipeline does not need to include OS hardening or server patching steps. However, the customer's PaaS responsibilities are significant: application code security, dependency management, secrets handling, and identity management. The practitioner designs a CI/CD pipeline that includes: SAST scanning (using Semgrep or SonarQube) to detect injection vulnerabilities, broken authentication, and insecure serialisation before every build; Software Composition Analysis to identify vulnerable open-source dependencies; a secrets scanning step to catch credentials committed to source code; and integration with Google Cloud's Secret Manager to replace any hardcoded configuration values. Additionally, the pipeline includes an automated check that the application's service account has only the minimum IAM permissions required. The practitioner explains to the development team that PaaS does not mean the application is automatically secure — it means the infrastructure layer is managed, and the team can and must invest that operational time directly into application security. The result is a pipeline where security is embedded at every stage and matched precisely to the responsibilities the PaaS model places on the customer.

Best Practices

- **Read and document the provider's shared responsibility model for every service before deployment.** Do not assume the responsibility boundary — read the provider's published documentation. For each service in use, create a two-column table: 'Provider responsibility' and 'Customer responsibility'. Share this with the team responsible for operating the service, require them to identify how each customer responsibility is addressed, and escalate any unaddressed responsibility to the CISO before the service goes live.
- **Never assume 'the cloud handles it' for any security control in an IaaS environment.** In IaaS, the customer owns everything above the hypervisor. OS patching, host firewall rules, storage encryption configuration, application security, and data protection are all customer responsibilities. Implement automated patch management (AWS Systems Manager Patch Manager, Azure Update Management, or equivalent), enforce configuration baselines at launch, and scan continuously for configuration drift. The Capital One breach is a standing reminder of what happens when this discipline lapses.
- **In PaaS environments, invest the operational overhead saved on infrastructure into application security.** PaaS removes the need to manage OS and runtime, which is a meaningful operational saving. That saving should be reinvested into a rigorous application security programme: OWASP Top 10 testing, dependency scanning, secrets management, and API security review. Organisations that treat PaaS as a reason to reduce security investment typically find that application vulnerabilities become their dominant risk.
- **Audit every available security feature in SaaS platforms and establish a configuration baseline.** SaaS platforms expose a wide range of security features — MFA enforcement, audit logging, DLP policies, session controls, conditional access — that are the customer's responsibility to enable and configure. Establish a

configuration baseline for each deployed SaaS platform using available CIS Benchmarks or the provider's security configuration guide. Review configurations on a quarterly schedule and use a CASB to provide continuous visibility and policy enforcement across the SaaS portfolio.

- **Scope compliance assessments by service category, not by organisation or environment as a whole.** When applying compliance frameworks such as HIPAA, PCI DSS, or SOC 2 to a cloud environment, scope the requirements to the service category in use for each workload. The compliance obligations of a HIPAA-covered entity using SaaS differ materially from those using IaaS. Applying IaaS-scoped controls to a SaaS environment wastes effort; failing to apply SaaS-scoped controls to a SaaS environment creates compliance exposure. Maintain a service inventory with compliance scope annotated by category.

Common Mistakes and Misconceptions

Misconception: Assuming that cloud providers handle security patching for virtual machines in IaaS environments.

Correct approach: In IaaS, operating system patching is explicitly the customer's responsibility. Implement automated OS patch management using the CSP's native patch management service or a third-party tool, enforce mandatory patching timelines through policy, and report on patch compliance monthly. Never assume the provider handles OS-level updates — the shared responsibility model documents are explicit on this point.

Misconception: Treating SaaS applications as fully secured because the provider manages the application code and infrastructure.

Correct approach: SaaS customers retain responsibility for identity and access management configuration, data governance, and enablement of available security features. Conduct a security configuration review of every SaaS platform in use, enable all relevant security features (audit logging, MFA, DLP), and establish a minimum security configuration baseline. The provider securing the application does not protect against customer misconfiguration.

Misconception: Applying identical security assessment methodology to IaaS, PaaS, and SaaS environments in the same organisation.

Correct approach: Scope the security assessment to match the customer responsibility boundary for each service category. IaaS assessments include OS hardening, patch management, and network configuration. PaaS assessments focus on application security and secrets management. SaaS assessments centre on configuration, access policy, and audit logging. Using a single methodology across all three categories will either over-assess controls the provider owns or under-assess controls the customer owns.

Misconception: Failing to re-read the shared responsibility model when a new service is added to the cloud environment.

Correct approach: The shared responsibility boundary can differ between services from the same provider — a managed database service and a raw virtual machine from the same CSP have different responsibility lines. Every time a new service is added, read the provider's shared responsibility documentation for that specific service, update the responsibility map, and confirm that all customer-side controls are assigned to a named owner.

table: Cloud Service Category Shared Responsibility Quick-Reference

Use this table to quickly determine which security controls belong to the customer versus the CSP for each service category. Always verify against the specific provider's published shared responsibility documentation for the service in use.

Security Control Area	IaaS	PaaS	SaaS
Physical security & data centre	CSP	CSP	CSP
Network hardware & infrastructure	CSP	CSP	CSP
Virtualisation / hypervisor	CSP	CSP	CSP
Operating system patching	Customer	CSP	CSP
Runtime environment patching	Customer	CSP	CSP
Application code security	Customer	Customer	CSP
Data encryption at rest	Customer	Shared	Shared
Data encryption in transit	Customer	Customer	Customer
Identity & access management	Customer	Customer	Customer
SaaS configuration & settings	N/A	N/A	Customer
Data classification & governance	Customer	Customer	Customer
Audit log enablement	Customer	Customer	Customer

How to use this: Entries marked 'Shared' indicate that responsibility is divided between provider and customer — the provider may encrypt data by default, but the customer is responsible for managing encryption keys and classification policies. Always read the provider's specific documentation to determine the exact division.

Knowledge Check

Test your understanding before moving on. Answers are shown below each question.

Q1. A healthcare organisation migrated patient record storage to AWS EC2 instances. Eight months after migration, an attacker exploited an unpatched Linux kernel vulnerability to gain root access to a server, resulting in a HIPAA breach. Which of the following best describes the accountability for this incident?

- A) A. AWS is fully accountable because the vulnerability was in the Linux kernel, which is an infrastructure component managed by the CSP
- B) B. The organisation is accountable because OS patching is a customer responsibility in IaaS environments, and AWS's shared responsibility model explicitly assigns this to the customer
- C) C. Accountability is shared equally between AWS and the organisation because both parties have security obligations in an IaaS model
- D) D. The organisation has no accountability because HIPAA compliance is a provider obligation when cloud services are used for protected health information

Answer: B. The organisation is accountable because OS patching is a customer responsibility in IaaS environments, and AWS's shared responsibility model explicitly assigns this to the customer

Q2. A development team is building a new application on Google App Engine (PaaS). A security review finds that database credentials are stored in environment variables accessible to the application runtime. What type of security failure does this represent, and who is responsible?

- A) A. An infrastructure security failure that is Google's responsibility because App Engine manages the underlying runtime environment

- B) B. A customer application-layer security failure — specifically a secrets management failure — because application code and configuration are the customer's responsibility in PaaS
- C) C. A shared responsibility failure because both the platform provider and the customer share accountability for secrets stored in environment variables
- D) D. A compliance failure that is automatically remediated when the application is deployed to a certified PaaS platform

Answer: B. A customer application-layer security failure — specifically a secrets management failure — because application code and configuration are the customer's responsibility in PaaS

Q3. An organisation uses Microsoft 365 for email and collaboration. A security audit finds that audit logging has not been enabled for the tenant, and users in the finance department have not been required to enrol in multi-factor authentication. Which statement correctly categorises these findings?

- A) A. Both findings are Microsoft's responsibility because Microsoft 365 is a SaaS platform and the provider manages all security controls
- B) B. Both findings are customer configuration responsibilities in a SaaS model, and represent gaps in the customer's security control implementation
- C) C. Audit logging is Microsoft's responsibility; MFA enforcement is the customer's responsibility
- D) D. Both findings are out of scope for cloud security assessment because SaaS applications are assessed only through the provider's SOC 2 report

Answer: B. Both findings are customer configuration responsibilities in a SaaS model, and represent gaps in the customer's security control implementation

Q4. Which of the following correctly describes the 'responsibility gap' in cloud security?

- A) A. The technical gap between a cloud provider's published security capabilities and the security controls actually deployed in customer environments
- B) B. The contractual gap between what a cloud provider promises in its service level agreement and what it delivers in practice
- C) C. The space between what customers assume the CSP covers and what the CSP actually covers under the shared responsibility model, which is a common cause of cloud security incidents
- D) D. The gap in security coverage that occurs when an organisation uses multiple cloud providers without a unified security management approach

Answer: C. The space between what customers assume the CSP covers and what the CSP actually covers under the shared responsibility model, which is a common cause of cloud security incidents

Q5. A Cloud Security Practitioner is designing security controls for a cloud environment that includes IaaS virtual machines, a PaaS managed database, and a SaaS CRM application. Which approach to control design is most appropriate?

- A) A. Apply a single unified security framework across all three service categories to ensure consistency of controls
- B) B. Apply the most restrictive control set (designed for IaaS) to all three environments to ensure comprehensive coverage
- C) C. Design separate control sets for each service category, scoped to the customer responsibility boundary in each, and documented against the shared responsibility model for each specific service
- D) D. Delegate control design for PaaS and SaaS to the respective providers and focus customer security resources exclusively on IaaS

Answer: C. Design separate control sets for each service category, scoped to the customer responsibility boundary in each, and documented against the shared responsibility model for each specific service

Reflection Questions

1. Think about a cloud service you currently use, either at work or personally. Which service category does it fall into? What specific security responsibilities do you hold as a user of that service?
2. Have you ever witnessed a 'responsibility gap' incident — where an organisation assumed a cloud provider handled a security control that actually belonged to the customer? What was the context, and what was the outcome?
3. The shared responsibility model shifts with every service category change. How would you communicate this to a non-technical business leader who is planning a cloud migration and assumes 'the cloud is secure'?
4. If your organisation migrated the same workload from IaaS to PaaS tomorrow, which of your current security controls would become the provider's responsibility? Which new controls would the customer need to implement that IaaS does not require?
5. How does understanding service categories change how you would structure a contract with a cloud provider? What specific provisions would you include in an IaaS contract that you would not need in a SaaS contract?

Key Takeaways

- IaaS gives the customer raw infrastructure resources; the customer is responsible for everything above the hypervisor including OS patching, middleware, application code, and data protection.
- PaaS shifts OS and runtime management to the provider; the customer's security responsibility concentrates on application code security, secrets management, and identity management.
- SaaS delivers a complete application; the customer's responsibility narrows to identity and access management, data governance, and configuration of the security features the provider makes available.
- The shared responsibility model is a formal, published document from every major CSP — reading it for each specific service in use is a non-negotiable step in cloud security governance.
- The responsibility gap — the mismatch between customer assumption and provider obligation — is the most common proximate cause of cloud security incidents and is closed only by explicitly mapping and operationalising the shared responsibility model.
- Multi-tenancy presents different risks by service category: hypervisor isolation failure in IaaS, runtime isolation failure in PaaS, and application data partitioning failure in SaaS.
- DevSecOps practices should be calibrated to the service category: IaaS requires OS hardening and patch automation; PaaS requires application SAST, SCA, and secrets management; SaaS requires configuration baseline management and CASB.
- Compliance requirements (HIPAA, PCI DSS, SOC 2) apply differently by service category — scope each compliance assessment to the specific customer responsibility boundary for the service category in use.

Recommended Next Actions

Put this lecture to work within the next seven days:

6. Locate and read the shared responsibility model documentation for the primary cloud provider your organisation uses — identify three specific controls you assumed were the provider's responsibility that are actually yours.
7. Create a cloud service inventory for your organisation listing each service, its service category (IaaS/PaaS/SaaS), the primary customer security responsibilities, and the current owner of each responsibility. Present this to your CISO as a shared responsibility map.
8. Review the OS patch compliance status of any IaaS virtual machines your organisation operates — confirm that an automated patching mechanism is in place and that patch compliance is being reported. Escalate any instances that are more than 30 days behind current patches.
9. Audit the security configuration of your organisation's primary SaaS platform against the relevant CIS Benchmark — enable any security features (audit logging, MFA, DLP) that are not currently active and document your findings as a baseline for future reviews.